

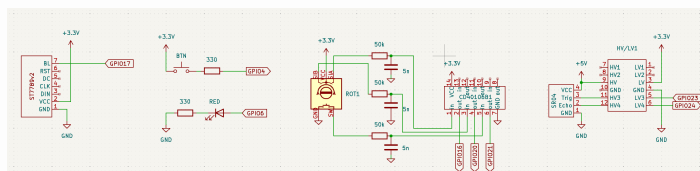
Linux w Systemach Wbudowanych

3. Wykorzystanie linii GPIO jako wejść w systemach Raspberry Pi i Linux

Piotr ZAWADZKI

Omówienie aplikacji

Schemat wszystkich połączeń elementów układu przedstawiono poniżej.



schemat układu

Funkcjonalnie aplikacja nie jest skomplikowana:

- dalmierz SR-04 w sposób ciągły mierzy odległość od przeszkody,
- gdy przeszkoda jest zbyt blisko, rozpoczynana jest procedura alarmu,
- wartość progu detekcji ustawia się za pomocą pokrętła enkodera obrotowego,
- działający alarm można dezaktywować, naciskając przycisk enkodera obrotowego,
- zmierzona odległość od przeszkody i ustawiony próg aktywacji alarmu są wyświetlane na ekranie,
- wartość odległości, która spowodowała wyzwolenie alarmu, jest wyświetlana na czerwono,
- w celach testowych można wyzwolić alarm, naciskając krótko przycisk chwilowy,
- długie naciśnięcie przycisku chwilowego kończy działanie aplikacji,
- działanie aplikacji można zakończyć kombinacją **Ctrl+C**. **Uwaga! Ctrl+C nie działa w terminalu Visual Studio Code.**

Każdy z elementów zarządzany jest przez osobny wątek.

```
std::thread lwsb_button_task( button_thread, std::ref(appState), std::vector<std::string>({ "/dev/inp",
std::thread detection_task( detection_thread, std::ref(appState), hardwareConfig.SR04_Trigger, hz
std::thread rotary_task( rotary_encoder_thread, std::ref(appState), hardwareConfig.rotary_SIA, ha
std::thread display_task( display_thread, std::ref(appState), hardwareConfig.displayConfig) ;
std::thread rotary_button_task( rotary_button_thread, std::ref(appState), hardwareConfig.rotary_S
std::thread alarm_led_task( gpio_led_thread, std::ref(appState), hardwareConfig.LED ) ;
std::thread displayBacklight_task( backlight_led_thread, std::ref(appState), hardwareConfig.displa
```

```
while (appState.keepRunning.load()) {
    std::this_thread::sleep_for(std::chrono::milliseconds(100));
    auto now = std::chrono::steady_clock::now();
    if (appState.setAlarm.load() && now > appState.alarmTime.load()) {
        std::cout << "ALARM expired" << std::endl;
        appState.setAlarm.store(false);
        appState.alarmTime.store(std::chrono::steady_clock::time_point::min());
    }
}
```

```
std::cout << "Main thread: waiting for child threads stop." << std::endl;
displayBacklight_task.join();
alarm_led_task.join();
rotary_button_task.join();
display_task.join();
rotary_task.join();
detection_task.join();
lwsb_button_task.join();
```

Wątki komunikują się ze sobą poprzez globalną zmienną

```

struct Application_state {
    std::atomic<bool> keepRunning;
    std::atomic<bool> setAlarm;
    std::atomic<std::chrono::steady_clock::time_point> alarmTime;
    std::atomic<float> distance; // in cm, distance measured by the sensor
    std::atomic<int> proximityThreshold; // in cm, distance below which the alarm is triggered
} appState = {
    .keepRunning = true,
    .setAlarm = false,
    .alarmTime = noalarmTime,
    .distance = 400.0,
    .proximityThreshold = 10
};

```

Jak widać jest to struktura złożona z niezależnych elementów atomicznych, które mogą być w bezpieczny sposób odczytywane i aktualizowane przez wątki.

Definicję sprzętowej konfiguracji wszystkich elementów zebrano w zmiennej `hardwareConfig`. Konfiguracja odpowiednich elementów jest przekazywana jako parametr do zainteresowanych wątków. Każdy wątek odpowiada za przydzielenie sobie zasobów i ich zwolnienie. W większości przypadków występujące w kodzie jawne zwalnianie zasobów może być pominięta, dzięki konstrukcji klas zarządzających zasobami zgodnie z zasadą RAII (*Resource Acquisition Is Initialization*), która w praktyce oznacza, że każdy zasób przydzielony w konstruktorze powinien być zwolniony w destruktorze. Destruktory są wykonywane zawsze, nawet gdy nastąpi rzucenie wyjątku (throw), więc stosowanie takiej zasady umożliwia bezpieczne zarządzanie zasobami.

Dzięki modułowej strukturze programu, kod procedur odpowiedzialnych za obsługę elementów jest samodzielny i jego analiza nie powinna nastręczać trudności.

Elementy wejściowe interfejsu aplikacji

- **GPIO 21**: Przycisk enkodera obrotowego zarządzany jako surowa linia GPIO z wykorzystaniem `libgpod` w wątku `rotary_button_thread`.
- **GPIO 4**: Przycisk zarządzany przez nakładkę `gpio-key` obsługiwany w wątku `button_thread`.
- **GPIO 16 i GPIO 20**: Linie wystawiane przez enkoder obrotowy i obsługiwane w wątku `rotary_encoder_thread`.
- **GPIO 24**: Linia *Echo* dalmierza obsługiwana w wątku `detection_thread` z wykorzystaniem klasy `SR04`.

Elementy wyjściowe interfejsu aplikacji

- **GPIO 23**: Linia *Trigger* dalmierza obsługiwana w wątku `detection_thread` z wykorzystaniem klasy `SR04`.
- **GPIO 17**: Podświetlenie ekranu sterowane za pomocą linii GPIO z wykorzystaniem `libgpod` w wątku `backlight_led_thread`.
- **GPIO 6**: dioda alarmowa zarządzana przez nakładkę `gpio-led` i sterowana w wątku `gpio_led_thread`.
- Wyświetlacz **ST7789v2** sterowany za pomocą magistrali SPI. Jego obsługa zostanie omówiona w jednym z kolejnych ćwiczeń. Za treści wyświetlane na ekranie odpowiada wątek `display_thread`.

Zadania do wykonania

0. Wstępne sprawdzenie konfiguracji linii GPIO za pomocą opcji `gpio=` w pliku `/boot/config.txt`

Opcja `gpio=` w pliku `/boot/config.txt` pozwala na ustawienie parametrów linii GPIO podczas uruchamiania systemu. Możemy zdefiniować kierunek (wejście lub wyjście), stan początkowy oraz rezystory podciągające dla konkretnych pinów GPIO. Dzięki temu linie są odpowiednio przygotowane, zanim system operacyjny zacznie z nich korzystać.

Sprawdź zgodność bieżącej konfiguracji (wykorzystaj polecenie `cat`) z konfiguracją opisaną poniżej.

Składnia opcji `gpio=`

Składnia polecenia jest następująca:

```
gpio=<numer_pinu>=<kierunek>,<rezystor>,<stan_początkowy>
```

- `<numer_pinu>`: Numer pinu GPIO (np. 4, 16, 20).
- `<kierunek>`: Kierunek pracy pinu:

- ip lub in — wejście (input).
- op lub out — wyjście (output).
- <rezystor>: Ustawienie rezystora podciągającego:
 - pu — podciągnięcie do VCC (pull-up).
 - pd — podciągnięcie do GND (pull-down).
 - pn — brak podciągnięcia (no pull).
- <stan_początkowy>: Stan początkowy dla wyjść:
 - dh — stan wysoki (drive high).
 - dl — stan niski (drive low).

Dodanie konfiguracji w /boot/config.txt

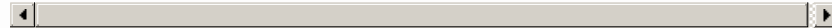
Aby odpowiednio skonfigurować linie GPIO wykorzystywane w ćwiczeniu, należy dodać poniższe linie do pliku /boot/config.txt:

```
# Wstępna konfiguracja linii GPIO
gpio=4=ip,pd
gpio=16=ip,pd
gpio=20=ip,pd
gpio=21=ip,pd
gpio=23=op,dh
gpio=24=ip,pd
```

Wyjaśnienie konfiguracji

- gpio=4=ip,pd: Ustawia GPIO 4 jako wejście z rezystorem pull-down.
- gpio=16=ip,pd i gpio=20=ip,pd: Ustawia GPIO 16 i 20 (enkloder obrotowy) jako wejścia z rezystorem pull-down.
- gpio=21=ip,pd: Ustawia GPIO 21 (przycisk enkodera) jako wejście z rezystorem pull-down.
- gpio=23=op,dh: Ustawia GPIO 23 (Trigger dalmierza HC-SR04) jako wyjście w stanie wysokim.
- gpio=24=ip,pd: Ustawia GPIO 24 (Echo dalmierza) jako wejście z rezystorem pull-down.
- Konfiguracja przycisku na GPIO 4 za pomocą gpio-key

```
dtoverlay=gpio-key,gpio=4,active_low=0,gpio_pull=down,label=button, keycode=16,debounce=200
```



Dlaczego warto użyć opcji gpio=?

- **Ustawienie stanów początkowych:** Zapewnia, że linie GPIO są w poprawnym stanie zaraz po uruchomieniu systemu.
- **Uniknięcie zakłóceń:** Poprawne podciągnięcie linii wejściowych zapobiega losowym zmianom stanu (plywaniu) sygnału.
- **Bezpieczeństwo urządzeń:** Zapobiega sytuacjom, w których nieprawidłowy stan linii GPIO mógłby uszkodzić podłączone urządzenia.

Uwagi dodatkowe

- **Kolejność wpisów:** Upewnij się, że wpisy gpio= są umieszczone przed innymi konfiguracjami, aby zostały zastosowane we właściwym momencie.
- **Zgodność sprzętowa:** Sprawdź dokumentację swojej platformy sprzętowej, aby upewnić się, że używane numery GPIO są poprawne i dostępne.
- **Restart systemu:** Po wprowadzeniu zmian w pliku /boot/config.txt należy zrestartować system, aby nowe ustawienia zostały zastosowane.

1. Wyświetlenie informacji o liniach GPIO

Polecenie:

Użyj narzędzia `gpiodetect` do wyświetlenia dostępnych w systemie kontrolerów GPIO oraz `gpioinfo` z pakietu **libgpiod** do wyświetlenia wszystkich dostępnych linii GPIO dla kontrolera `pinctrl-rp1`.

Wykonaj zrzut/zrzuty ekranu (3.1.x) zawierające wynik działania komend. Zrzuty zamieść w sprawozdaniu.

2. Monitorowanie zdarzeń przycisku na GPIO 4

Polecenie:

Uruchom evtest:

evtest

Zobaczysz listę dostępnych urządzeń wejściowych. Wybierz urządzenie odpowiadające przyciskowi (np.

event0

lub inne z nazwą gpio_keys lub button).

Przykład:

```
No device specified, trying to scan all of /dev/input/event*
Available devices:
/dev/input/event0: gpio_keys
Select the device event number [0-...]: 0
```

Naciskaj i zwalnij przycisk podłączony do GPIO 4, obserwując generowane zdarzenia w terminalu.

Wykonaj zrzut ekranu (3.2.1) zawierający wynik działania komendy. Zrzut zamieść w sprawozdaniu.

3. Monitorowanie stanu przycisku enkodera na GPIO 21

Polecenie:

Użyj gpiomon do monitorowania zboczy na GPIO 21. Wykonaj komendy monitorujące a) tylko zbocza narastające, b) tylko zbocza opadające, c) oba rodzaje zboczy. W czasie aktywnej komendy naciskaj przycisk enkodera, aby sprawdzić jej działanie.

```
gpiomon <opcje> <chip> <lineNum>
```

Składnia dostępna po gpiomon -h lub man gpiomon.

Wykonaj zrzut/zrzuty ekranu (3.3.x) zawierający wynik działania komendy. Zrzut zamieść w sprawozdaniu.

4. Badanie sygnałów enkodera obrotowego na GPIO 16 i GPIO 20

Polecenie:

Monitoruj stan linii 16 i 20 enkodera obrotowego korzystając z gpiomon. Przyjmij wyzwalanie zboczami narastającym i opadającym. *Wskazówka* gpiomon pozwala na monitorowanie stanu wielu linii. Wykonaj kilka przejść enkodera w prawo (CW), zatrzymaj komendę, a następnie kilka w lewo (CCW). Porównaj otrzymane sekwencje.

Wykonaj zrzut/zrzuty ekranu (3.4.x) zawierający wynik działania komendy. Zrzut zamieść w sprawozdaniu.

5. Pomiar odległości z linii komend za pomocą dalmierza HC-SR04

Polecenia:

0. Ustaw przeszkodę przed dalmierzem.
1. Otwórz dwie sesje na system target.
2. W sesji A ustaw GPIO 23 jako wyjście i ustaw stan niski *Trigger*:

```
gpioset gpiochip0 23=0
```
3. W sesji B sprawdź, czy *Echo* jest w stanie niskim:

```
gpioget gpiochip0 24
```
4. W sesji B włącz monitorowanie obu zboczy, ogranicz się do dwóch zdarzeń, wykorzystaj opcję formatu postaci --format="%e %s.%n", pytajniki zastąp

odpowiednimi opcjami

```
gpiomon ??? --format="%e %s.%n" gpiochip0 24
```

5. W sesji A wywołaj sekwencję pomiaru komendą

```
gpioset --mode=time --usec 20 gpiochip0 23=1 && gpioset gpiochip0 23=0
```

6. W sesji A zanotuj różnicę czasu w mikrosekundach. Oblicz odległość od przeszkody wg wzoru

$$D[m] = 0.5 * t[\mu s] * 10^{-6} * 340[m/s]$$

Wykonaj zrzut ekranu (3.5.1) zawierający wynik działania komendy. Zrzut oraz wyliczoną odległość zamieść w sprawozdaniu.

6. Badanie ustawień pinów za pomocą pinctrl i gpioinfo

Uwaga Główna różnica pomiędzy gpioinfo a pinctrl polega na tym, że gpioinfo komunikuje się ze sterownikiem kontrolera linii, natomiast pinctrl bada/modyfikuje bezpośrednio zawartość rejestrów. Używanie pinctrl do modyfikacji nastaw może prowadzić do niestabilności systemu, bo wprowadza niezgodność pomiędzy rzeczywistym stanem rejestrów a tym, co sobie o nich myśli sterownik systemu operacyjnego.

Polecenia:

1. Wyświetl aktualną konfigurację pinów:

```
ls /sys/kernel/debug/pinctrl/  
cat /sys/kernel/debug/pinctrl/*-rp1/pins
```

2. Sprawdź ustawienia pinów wykorzystywanych w ćwiczeniu

```
pinctrl get 4,6,16,17,20,21,23,24  
gpioinfo gpiochip0
```

3. W sesji A uruchom aplikację demo

4. w sesji B sprawdź ponownie ustawienia pinów. W rezultatach komendy gpioinfo zwróć uwagę na nazwy przypisane do pinów.

Wykonaj zrzut ekranu (3.6.1) zawierający wynik działania komendy z punktu 4. Zrzut zamieść w sprawozdaniu.

7. Praca z kodem aplikacji

Polecenie

Zbuduj dostarczony kod źródłowy. Jest to aplikacja nieco okaleczona w porównaniu do dostarczonej wersji demo. Celem dalszych działań jest przywrócenie pierwotnej funkcjonalności. Miejsca wymagające modyfikacji oznaczono jako TODO. Stosuj się do opisów zamieszczonych w komentarzach. Przed wykonaniem zrzutu ekranu nie kasuj komentarzy zawierających polecenie.